

- Description

This module manages a smart swimming pool system, ensuring that:

1. The pH level of the pool water is continuously monitored (as a numerical value).
2. If the pH level falls below the optimal range (e.g., below 7.2), the system automatically triggers the dosing of alkaline chemicals to raise the pH.
3. If the pH level exceeds the optimal range (e.g., above 7.8), the system automatically triggers the dosing of acidic chemicals to lower the pH.
4. If the pH level is within the optimal range, no chemical dosing is performed.

This system helps maintain the ideal water quality in the swimming pool, ensuring safety and comfort for swimmers.

- Specification

module SmartPoolPHControl

var currentPH, optimalLow, optimalHigh: real

process updatePH(newPH: real) returns (currentPH: real)

functional scenario form:

currentPH < optimalLow && output "ALERT: pH is too low (" + currentPH + ").
Dosing alkaline chemicals to raise pH.", pH is below optimal, trigger alkaline dosing.

||

currentPH > optimalHigh && output "ALERT: pH is too high (" + currentPH + ").
Dosing acidic chemicals to lower pH.", pH is above optimal, trigger acidic dosing.

||

currentPH <= optimalHigh && output "pH is optimal (" + currentPH + "). No
chemical dosing required.", pH is within optimal range.

end process

end module

- Java Code

```
package code;
public class c83_SmartPool {
    private double optimalLow;
    private double optimalHigh;
    private double currentPH;
    public c83_SmartPool(double optimalLow, double optimalHigh, double
initialPH) {
        this.optimalLow = optimalLow;
        this.optimalHigh = optimalHigh;
        this.currentPH = initialPH;
    }
    public void updatePH(double newPH) {
        currentPH = newPH;
        System.out.printf("Current pH: %.2f%n", currentPH);
        if (currentPH < optimalLow) {
```

```

        System.out.println("ALERT: pH is too low. Dosing alkaline c
hemicals to raise pH.");
    } else if (currentPH > optimalHigh) {
        System.out.println("ALERT: pH is too high. Dosing acidic ch
emicals to lower pH.");
    } else {
        System.out.println("pH is optimal. No chemical dosing requi
red.");
    }
}
public double getCurrentPH() {
    return currentPH;
}
public static void main(String[] args) {
    c83_SmartPool pool = new c83_SmartPool(7.2, 7.8, 7.5);
    // testcase-VT:
//    pool.updatePH(6.8);
//    pool.updatePH(7.5);
//    pool.updatePH(8.0);

    // testcase-FT:
//    pool.updatePH(-1.7976931348623157E308);
//    pool.updatePH(1.0622392272425998E291);
//    pool.updatePH(2.7308318922745074E293);

    // testcase-Z3:
//    pool.updatePH(6.8);
//    pool.updatePH(7.5);
//    pool.updatePH(8.0);

    // testcase-UVT:
    pool.updatePH(6.8);
    pool.updatePH(7.5);
    pool.updatePH(8.0);
}
}

```

- Coverage report

1. Test case number: 3
2. V-Method testing branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
✓ VibrationTestingTool	0.5 %	4	838	842
✓ src	0.5 %	4	838	842
✓ code	0.5 %	4	838	842
✓ c83_SmartPool.java	100.0 %	4	0	4
✓ c83_SmartPool	100.0 %	4	0	4
main(String[])		0	0	0
c83_SmartPool(double, double,)		0	0	0
updatePH(double)	100.0 %	4	0	4

3. Vibration testing instruction coverage: 95.8%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
✓ VibrationTestingTool	0.3 %	68	21,113	21,181
✓ src	0.3 %	68	21,113	21,181
✓ code	0.3 %	68	21,113	21,181
✓ c83_SmartPool.java	95.8 %	68	3	71
✓ c83_SmartPool	95.8 %	68	3	71
main(String[])	100.0 %	17	0	17
c83_SmartPool(double, double,)	100.0 %	12	0	12
updatePH(double)	100.0 %	39	0	39

4. Fuzz testing branch coverage: 75%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
✓ VibrationTestingTool	0.4 %	3	839	842
✓ src	0.4 %	3	839	842
✓ code	0.4 %	3	839	842
✓ c83_SmartPool.java	75.0 %	3	1	4
✓ c83_SmartPool	75.0 %	3	1	4
main(String[])		0	0	0
c83_SmartPool(double, double,)		0	0	0
updatePH(double)	75.0 %	3	1	4

5. Fuzz testing instruction coverage: 91.5%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
✓ VibrationTestingTool	0.3 %	65	21,116	21,181
✓ src	0.3 %	65	21,116	21,181
✓ code	0.3 %	65	21,116	21,181
✓ c83_SmartPool.java	91.5 %	65	6	71
✓ c83_SmartPool	91.5 %	65	6	71
main(String[])	100.0 %	17	0	17
c83_SmartPool(double, double,)	100.0 %	12	0	12
updatePH(double)	92.3 %	36	3	39

6. Z3 branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
✓ VibrationTestingTool	0.5 %	4	838	842
✓ src	0.5 %	4	838	842
✓ code	0.5 %	4	838	842
✓ c83_SmartPool.java	100.0 %	4	0	4
✓ c83_SmartPool	100.0 %	4	0	4
main(String[])		0	0	0
c83_SmartPool(double, double,)		0	0	0
updatePH(double)	100.0 %	4	0	4

7. Z3 instruction coverage: 95.8%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
✓ VibrationTestingTool	0.3 %	68	21,113	21,181
✓ src	0.3 %	68	21,113	21,181
✓ code	0.3 %	68	21,113	21,181
✓ c83_SmartPool.java	95.8 %	68	3	71
✓ c83_SmartPool	95.8 %	68	3	71
main(String[])	100.0 %	17	0	17
c83_SmartPool(double, double,)	100.0 %	12	0	12
updatePH(double)	100.0 %	39	0	39

8. Unimproved V-method branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	0.5 %	4	838	842
▼ src	0.5 %	4	838	842
▼ code	0.5 %	4	838	842
▼ c83_SmartPool.java	100.0 %	4	0	4
▼ c83_SmartPool	100.0 %	4	0	4
main(String[])		0	0	0
c83_SmartPool(double, double,)		0	0	0
updatePH(double)	100.0 %	4	0	4

9. Unimproved V-method instruction coverage: 95.8%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	0.3 %	68	21,113	21,181
▼ src	0.3 %	68	21,113	21,181
▼ code	0.3 %	68	21,113	21,181
▼ c83_SmartPool.java	95.8 %	68	3	71
▼ c83_SmartPool	95.8 %	68	3	71
main(String[])	100.0 %	17	0	17
c83_SmartPool(double, double,)	100.0 %	12	0	12
updatePH(double)	100.0 %	39	0	39